

Progetto: Sentiment Analysis

Sintesi degli obiettivi

Il presente progetto si pone l'obiettivo di sviluppare un sistema di **Sentiment Analysis** applicata ad un corpus di tweet generici, finalizzato alla classificazione automatica sull'opinione espressa nei testi. L'intento principale è quello di valutare l'efficacia di tecniche di Natural Language Processing (NLP) nel trattamento di un linguaggio informale, breve e rumoroso, tipico dei social media. Oltre all'analisi del contesto testuale, il progetto si propone di integrare l'analisi delle emoji, considerate elementi fondamentali della comunicazione digitale per l'espressione di emozioni, stati d'animo e intenzioni comunicative. Questa scelta metodologica mira a evitare la perdita di informazioni semantiche rilevanti durante la fase di pulizia e pre-processing del testo, migliorando così la capacità del sistema di cogliere il sentiment complessivo del messaggio.

Risorse utilizzate

Nello sviluppo del modello sono state impiegate le seguenti risorse:

- **Libreria re (Regular Expression)**: utilizzata per la pulizia del rumore testuale tramite pattern matching, consentendo la rimozione di URL, menzioni e simboli non informativi;
- **Libreria pandas**: utilizzata per caricare il dataset in memoria ed effettuare operazioni su di esso in maniera molto più efficiente rispetto ad usare dei semplici oggetti Python;
- **Libreria emoji**: fondamentale per il task di demojizzazione, permettendo la conversione delle emoji in token testuali semanticamente interpretabili;
- **Libreria pandarallel**: utilizzata per ottimizzare i tempi di computazione, applicando tutte le funzioni di preprocessing in parallelo sui dataframe Pandas.
- **Libreria NLTK**: utilizzata per la tokenizzazione specifica per Twitter, la rimozione di stopwords, la lemmatizzazione basata su WordNet e l'analisi lessicale del sentiment tramite VADER;
- **Libreria Scikit-learn**: utilizzata per la vettorizzazione (TF-IDF), la suddivisione del dataset (`train_test_split`), l'implementazione dei classificatori (MultinomialNB, ComplementNB e LogisticRegression) e la valutazione delle prestazioni tramite `classification_report`, `accuracy_score` e `confusion matrix`;
- **Libreria Scipy**: utilizzo di `hstack` per la fusione di differenti vettori di feature in un'unica matrice sparsa;
- **Libreria Seaborn e Matplotlib**: utilizzate per la visualizzazione tramite grafici e heatmaps, per facilitare l'interpretazione visiva delle performance e degli errori del modello;

Sviluppo del progetto

1. Preprocessing del testo

Il corpus iniziale, costituito da circa 3.500.000 tweet, è stato sottoposto a un accurato processo di pulizia e normalizzazione per ridurre il rumore tipico del linguaggio informale sui social media. In particolare, le operazioni effettuate sono state:

- **Rimozione di URL, retweet, menzioni (@username) e numeri, limitazione dei caratteri ripetuti a tre e pulizia degli hashtag (manteniamo solo il testo dell'hashtag)** tramite l'utilizzo di espressioni regolari per eliminare gli elementi non rilevanti per il contenuto semantico;
- **Demojizzazione**, ossia la trasformazione delle emoji in token testuali, per preservare le informazioni emotive trasmesse attraverso le emoji;
- **Tokenizzazione** specifica per Twitter tramite **TweetTokenizer** di NLTK, in grado di gestire correttamente abbreviazioni, emoticon e punteggiatura informale;
- **Lemmatizzazione basata su WordNet**, per ridurre la variabilità morfologica delle parole, e migliorare la capacità di generalizzazione del modello;
- **Rimozione delle stopwords** in lingua inglese, con l'obiettivo di ridurre il rumore lessicale;
- **Gestione delle negazioni** così da evitare che termini come "NOT" perdano il loro effetto semantico durante le fasi di tokenizzazione e vettorizzazione;

2. Rappresentazione delle feature

Per convertire i tweet in una rappresentazione numerica adatta all'addestramento dei classificatori, è stata utilizzata una codifica TF-IDF basata su unigram, bigram e trigram, in grado di catturare sia la frequenza delle singole parole sia alcune combinazioni lessicali significative. Questa scelta ha permesso di modellare sia la frequenza delle singole parole sia alcune combinazioni lessicali rilevanti, catturando parzialmente il contesto locale del testo. L'utilizzo del peso TF-IDF permette inoltre di ridurre l'impatto dei termini molto frequenti ma scarsamente informativi, valorizzando, al contrario, quelli più discriminativi per il compito di classificazione ai fini della classificazione del sentiment.

Rispetto ad una rappresentazione Bag of Words basata su semplici conteggi, l'approccio TF-IDF fornisce una descrizione più informativa e robusta dei testi, migliorando la capacità del modello di distinguere correttamente le diverse classi di sentiment.

Accanto alla rappresentazione testuale, sono state introdotte feature di sentiment a livello globale. In particolare, lo score di sentiment del testo è stato calcolato utilizzando VADER di NLTK, mentre il contributo emotivo delle emoji è stato modellato tramite uno score derivato da un dataset che riporta, per ciascuna emoji, il numero di occorrenze Negative, Positive e Neutral; lo score finale è stato ottenuto come media normalizzata di tali valori. In fase di progettazione, si è scelto di attribuire un peso maggiore al contributo delle emoji rispetto a quello testuale, qualora fossero presenti nel tweet, sulla base dell'assunzione che le emoji veicolino un segnale emotivo più diretto e meno ambiguo.

3. Modelli di classificazione

Nel progetto sono stati sperimentati diversi modelli di classificazione, selezionati per la loro efficacia nel trattamento di dati testuali ad alta dimensionalità:

- **Multinomial Naive Bayes (MultinomialNB):** modello di classificazione che assume l'indipendenza condizionale delle feature e stima la probabilità di appartenenza a ciascuna classe sulla base delle frequenze dei termini. Esso risulta particolarmente adatto alla classificazione del testo vettorizzato tramite TF-IDF;
- **Complement Naive Bayes (ComplementNB):** variante di Naive Bayes progettata per gestire i dataset sbilanciati, in cui la distribuzione delle classi non è uniforme. Il modello utilizza le statistiche delle classi complementari per ridurre il bias verso la classe dominante, migliorando le prestazioni in scenari con forte imbalance;
- **Logistic Regression:** modello discriminante lineare che ottimizza direttamente la separazione tra classi mediante la massimizzazione della log-likelihood. Questo approccio si dimostra particolarmente efficace su grandi dataset e in combinazione con rappresentazioni TF-IDF, offrendo un buon compromesso tra accuratezza e interpretabilità

4. Valutazione e metriche

La valutazione delle prestazioni dei modelli di classificazione è stata condotta utilizzando diverse metriche standard:

- **Accuracy globale**, utilizzata per misurare la percentuale complessiva di predizioni corrette sul dataset di test. Questa metrica può risultare fuorviante in presenza di classi sbilanciate;
- **Precision, Recall e F1-score** calcolate per ciascuna classe, al fine di analizzare in modo più approfondito il compromesso tra falsi positivi e falsi negativi. In particolare, F1-score consente di sintetizzare precision e recall in un'unica misura, risultando particolarmente utile in scenari con distribuzione delle classi non uniforme;
- **Confusion Matrix**, impiegata per analizzare nel dettaglio gli errori di classificazione, permettendo di osservare quali classi vengono maggiormente confuse tra loro;
- **Analisi della distribuzione delle classi** effettuata per tenere conto di eventuali squilibri nel dataset e per interpretare correttamente le metriche ottenute, evitando valutazioni distorte delle prestazioni dei modelli;

5. Analisi

In questa fase finale, vengono analizzati e messi a confronto i risultati ottenuti dall'applicazione dei tre diversi algoritmi di Machine Learning sopra elencati.

L'obiettivo è la valutazione dell'accuratezza complessiva di ciascun modello nella classificazione del sentiment e la verifica della robustezza dei classificatori di fronte ad un dataset sbilanciato, dove la classe Negative rappresenta un numero di campioni significativamente inferiori rispetto alle altre.

Multinomial Naive Bayes:

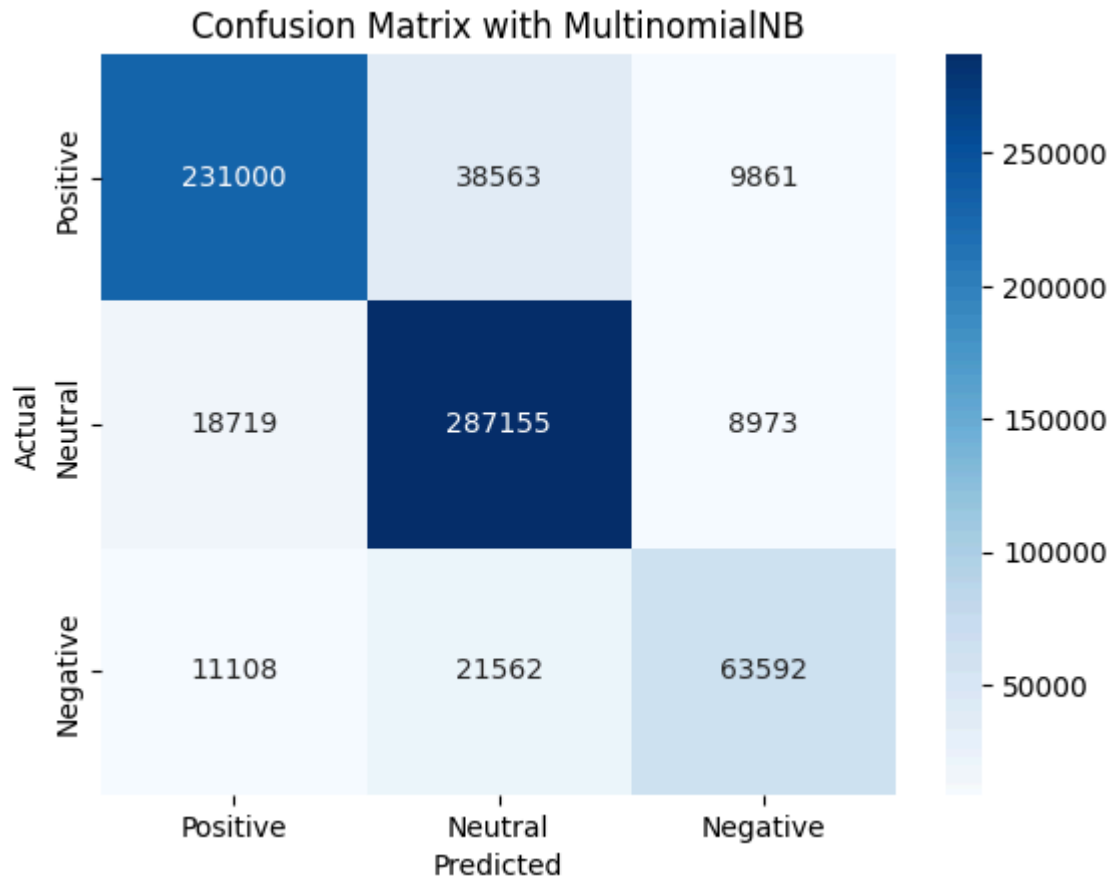
Global Accuracy with MultinomialNB: 0.84

	precision	recall	f1-score	support
Negative	0.77	0.66	0.71	96262
Neutral	0.83	0.91	0.87	314847
Positive	0.89	0.83	0.86	279424
accuracy			0.84	690533
macro avg	0.83	0.80	0.81	690533
weighted avg	0.84	0.84	0.84	690533

Il modello ha ottenuto un accuracy globale dell'84% su un dataset totale di 690.533 campioni. Questo indica una solida capacità predittiva generale, considerando che si tratta di un problema di classificazione a tre classi (Positive, Neutral, Negative).

Precision e Recall delle rispettive classi:

- Classe Neutral: è la classe con i risultati migliori, con un recall del 91%. il modello è estremamente bravo a identificare i casi neutri, lasciandosi sfuggire solo il 9%;
- Classe Positive: presenta la precision più alta (89%). Questo valore attesta l'affidabilità del modello nelle predizioni positive;
- Classe Negative: al contrario delle precedenti, questa classe mostra performance inferiori, evidenziando la tendenza del modello a confondere una parte dei messaggi negativi con quelli neutri, probabilmente a causa della natura sbilanciata del dataset.



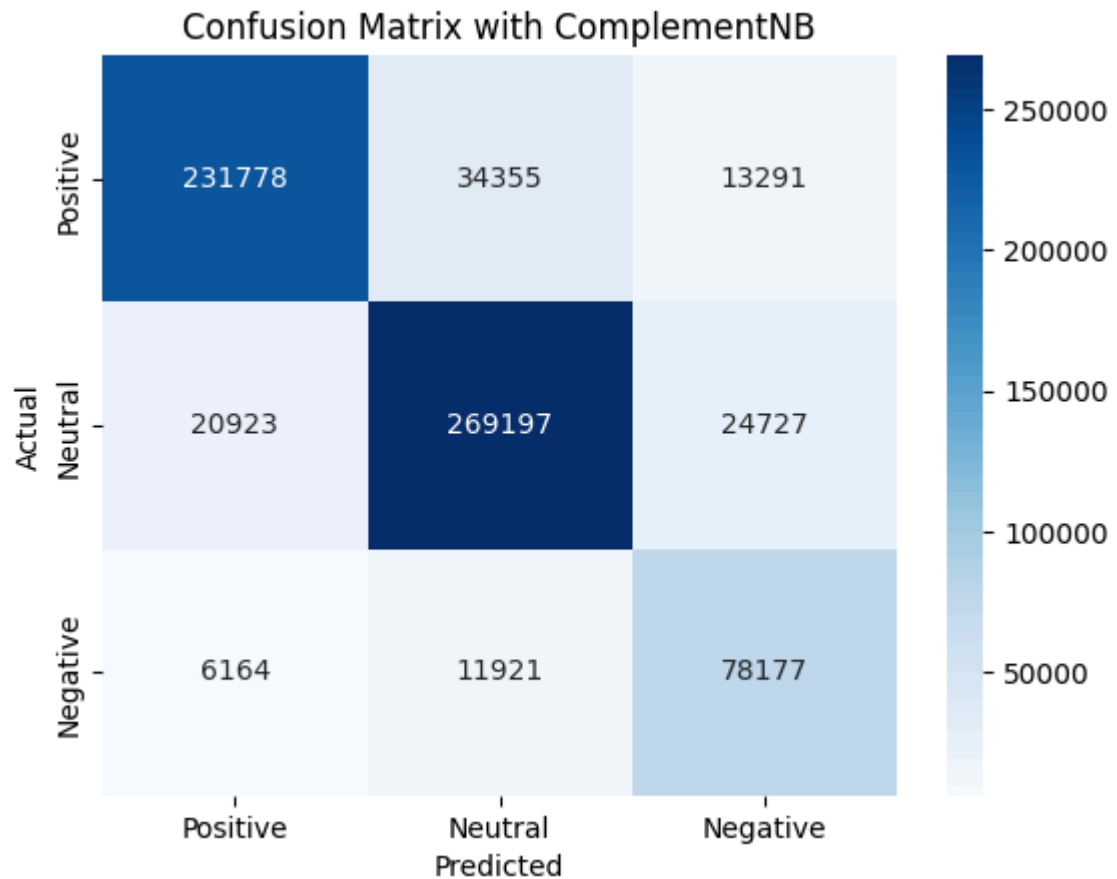
Poiché il classificatore calcola le probabilità a priori basandosi sulla frequenza delle classi, se la classe “Neutral” è molto frequente nel dataset di addestramento, il modello tenderà a “scommettere” su di essa nei casi di incertezza. Un fattore dominante che spiega la “fuga” dei dati verso la classe Neutral è l’incapacità intrinseca dei modelli NB di interpretare le figure retoriche complesse, in particolare il sarcasmo. Questa mancanza crea un addensamento sulla colonna centrale.

Complement Naive Bayes:

Global Accuracy with ComplementNB: 0.84				
	precision	recall	f1-score	support
Negative	0.67	0.81	0.74	96262
Neutral	0.85	0.86	0.85	314847
Positive	0.90	0.83	0.86	279424
accuracy			0.84	690533
macro avg	0.81	0.83	0.82	690533
weighted avg	0.85	0.84	0.84	690533

Usando questo classificatore notiamo un miglioramento nella classe Negative:

- Recall dell'81%, indica che il modello ora riesce a identificare l'81% di tutti i commenti negativi reali;
- Precision del 67% significa che quando il modello predice Negative, la previsione è corretta 2 volte su 3. Molti casi vengono etichettati come negative quando forse non lo sono;



La diagonale principale mostra un alto numero di Veri Positivi per tutte le classi, confermando l'efficacia del classificatore. Nonostante la classe Negative sia la meno rappresentata nel dataset, a causa dello sbilanciamento dei dati, il modello riesce ad identificarla correttamente. Si può osservare anche una frequenza molto bassa di errori tra classi opposte, ciò indica che il modello distingue bene le polarità forti.

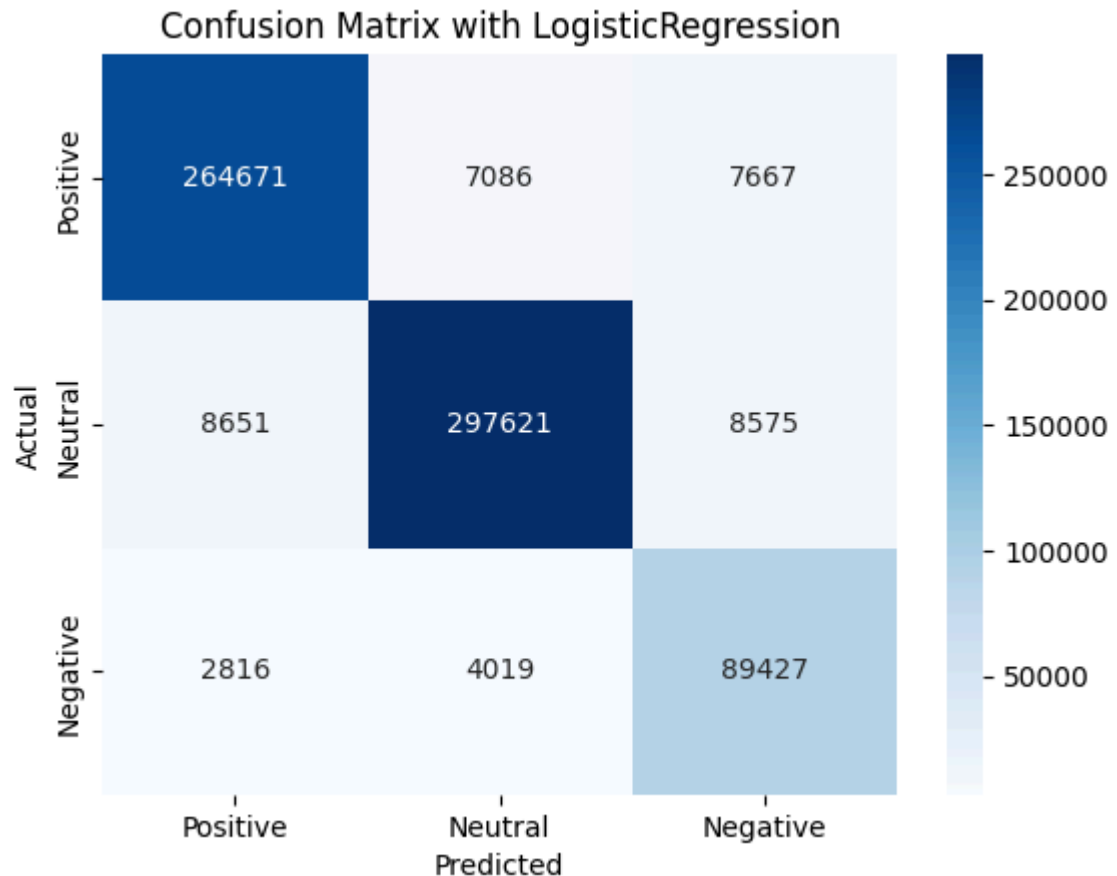
Logistic Regression:

Global Accuracy with LogisticRegression: 0.94

	precision	recall	f1-score	support
Negative	0.85	0.93	0.89	96262
Neutral	0.96	0.95	0.95	314847
Positive	0.96	0.95	0.95	279424
accuracy			0.94	690533
macro avg	0.92	0.94	0.93	690533
weighted avg	0.95	0.94	0.94	690533

Il modello dimostra una capacità di generalizzazione superiore, con performance bilanciate tra le diverse categorie:

- Neutral e Positive mostrano punteggi eccellenti con una precision del 96%, che indica quanto il modello sbaglia raramente;
- Negative: Nonostante sia la classe con il minor support, il modello ottiene una recall molto alta (93%). Ciò significa che la Logistic Regression è estremamente efficace nell'identificare i tweet negativi, minimizzando i falsi negativi;



Dalla confusion matrix notiamo che il modello ha classificato correttamente i veri positivi e, rispetto ai modelli precedenti, la “dispersione” dei dati fuori dalla diagonale è drasticamente ridotta. L’area dell’incertezza rimane, seppur ridotta, tra le classi adiacenti, suggerendo che alcune sfumature linguistiche rimangono intrinsecamente ambigue.

6. Conclusioni

L’analisi comparativa tra i modelli testati ha evidenziato una netta superiorità delle Logistic Regression rispetto alle varianti del Naive Bayes analizzate. Sebbene la Multinomial Naive Bayes rappresenti l’algoritmo di riferimento standard per la classificazione testuale, la scelta di adottare il Complement Naive Bayes è stata dettata dalla sua specifica capacità di lavorare con la presenza di classi sbilanciate. Tuttavia, i dati confermano che la Logistic Regression ha elevato ulteriormente le prestazioni complessive, raggiungendo un accuracy globale del 94%. Questo risulta particolarmente evidente osservando la precision del 96% ottenuta sulle classi Negative e Neutral, che testimoniano un’altissima affidabilità del modello nell’interpretare correttamente le polarità dominanti del dataset. Anche sulla classe

minoritaria, Negative, il modello ha dimostrato una robustezza superiore sia alla Multinomial che al Complement, riuscendo a identificare la quasi totalità delle critiche, nonostante il minor numero di esempi a disposizione.

Guardando al futuro, il progetto offre diverse opportunità di evoluzione per affinare ulteriormente la capacità di analisi semantica. Un primo passo potrebbe essere l'integrazione di n-grammi più complessi per catturare al meglio le negazioni e le sfumature di significato che ancora generano una leggera incertezza tra le classi contigue. Un altro passo potrebbe essere l'adozione di architetture Deep Learning basate su Transformer, permettendo al modello di comprendere il contesto linguistico con una profondità superiore rispetto agli approcci lineari e probabilistici attuali.

Link alle risorse utilizzate

Dataset:

<https://www.kaggle.com/datasets/sandeepkumarkushwaha/t4sa-dataset/data>

pandas: <https://pandas.pydata.org/>

emoji: <https://github.com/carpedm20/emoji/>

pandarallel: <https://nalepae.github.io/pandarallel/>

contractions: <https://github.com/kootenpv/contractions>

nltk: <https://www.nltk.org/>

scikit-learn: <https://scikit-learn.org/stable/>

scipy: <https://scipy.org/>

seaborn: <https://seaborn.pydata.org/>

matplotlib: <https://matplotlib.org/>

Divisione dei compiti

- Giulia Filicicchia: Scelta del dataset, preprocessing e valutazione delle feature
- Samuele Maria Morreale: Scelta e implementazione dei modelli, presentazione dei risultati ottenuti